

List OF Question of slide 5

1. Compare connectionless and connection oriented services

Comparison Between Connectionless and Connection-Oriented Services

Aspect	Connection-Oriented Service	Connectionless Service
Definition	A communication method where a connection is established before data transfer.	A communication method where data is sent without establishing a dedicated connection.
Setup	Requires a connection setup before communication (e.g., TCP 3-way handshake).	No setup required, packets are sent independently.
Reliability	Ensures reliable delivery with error checking and retransmission.	Does not guarantee reliable delivery; data may be lost or arrive out of order.
Overhead	Higher overhead due to connection establishment and maintenance.	Lower overhead as no connection setup is required.
Packet Ordering	Maintains the correct order of packets.	Packets may arrive out of order and need reordering at the receiver's end.
Examples	TCP (Transmission Control Protocol), ATM (Asynchronous Transfer Mode).	UDP (User Datagram Protocol), ICMP (Internet Control Message Protocol).
Use Cases	Suitable for applications requiring reliability (e.g., file transfer, web browsing, emails).	Suitable for real-time applications where speed is important (e.g., VoIP, video streaming, online gaming).

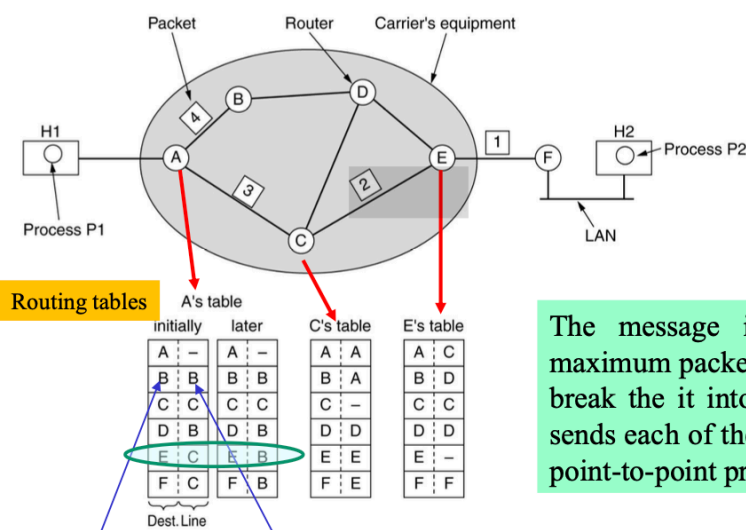
Key Takeaways

- **Connection-oriented services** (e.g., TCP) **ensure reliability and ordered delivery but introduce extra overhead.**
- **Connectionless services** (e.g., UDP) **prioritize speed and efficiency at the cost of reliability.**

Would you like an example implementation of TCP and UDP in Python? 🚀

2. Example of connectionless service routing

Suppose that the process P1 in fig. has a long message for P2. It hand the message to the transport layer with instructions to deliver it to process P2 on host H2. The transport layer code runs on H1, typically within the operating system. It prepends (add something) a transport header to the front of the message and hands the result to the network layer, probably just another procedure within the operating system.



Connectionless Service

The message is four times longer than the maximum packet size, so the network layer has to break the it into four packets 1, 2, 3 and 4 and sends each of them in turn to router A using some point-to-point protocol.

Destination

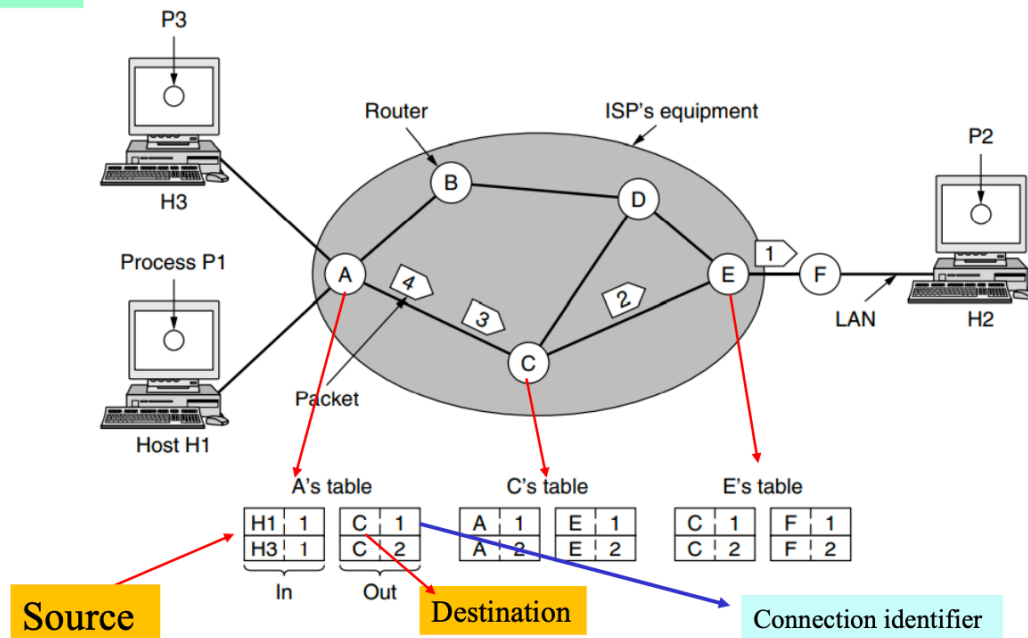
Via which node

Routing of datagram in a network

6

3. Example of connection oriented service routing

- ✓ Here, host H1 has established connection 1 with host H2. This connection is remembered as the first entry in each of the routing tables.
- ✓ The first line of A's table says that if a packet bearing **connection identifier 1** comes in from H1, it is to be sent to router C and given **connection identifier 1**. Similarly, the first entry at C routes the packet to E, also with **connection identifier 1**.



9

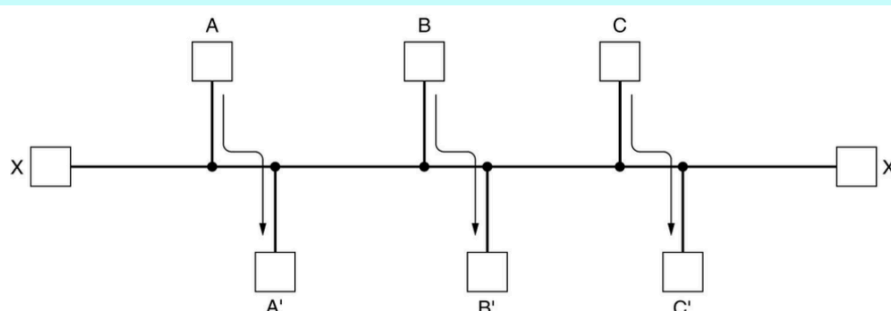
4. Compare datagram subnet and virtual circuit subnet

Comparison of Virtual-Circuit and Datagram Subnets

Issue	Datagram subnet	Virtual-circuit subnet
Circuit setup	Not needed	Required
Addressing	Each packet contains the full source and destination address	Each packet contains a short VC number
State information	Routers do not hold state information about connections	Each VC requires router table space per connection
Routing	Each packet is routed independently	Route chosen when VC is set up; all packets follow it
Effect of router failures	None, except for packets lost during the crash	All VCs that passed through the failed router are terminated
Quality of service	Difficult	Easy if enough resources can be allocated in advance for each VC
Congestion control	Difficult	Easy if enough resources can be allocated in advance for each VC ¹¹

5. How fairness and optimally conflict with each other. Show with an example

Fairness and **optimally** may sound obvious-surely no reasonable person would oppose them-but as it turns out, they often contradictory goals. Suppose that there is enough traffic between A and A', between B and B', and between C and C' to saturate the horizontal links. To maximize the total flow, the X and X' traffic should be shut off altogether. Unfortunately, X and X' may not see it that way. Evidently **some compromise** between global **efficiency** and **fairness** to individual connections is needed.



Conflict between fairness and optimality.

In context of **optimality**, we actually need to reduce delay of packet. The routing algorithm attempt to minimize the number of hops to reduce delay and increase the throughput.

Explanation of the Image: Conflict Between Fairness and Optimality in Routing

The image and text explain a fundamental trade-off in networking: **Fairness vs. Optimality** in routing decisions.

Key Concepts

1. Fairness vs. Optimality Conflict

- **Fairness** means distributing network resources equally among all users and connections.
- **Optimality** focuses on maximizing total throughput and reducing delays, even if it means prioritizing some connections over others.

2. Network Scenario in the Diagram

- The network consists of **three main traffic flows**:
- **A → A'**, **B → B'**, and **C → C'** (along horizontal links).
- Additionally, **X → X'** is another traffic flow using the same network.

- If traffic on $A \rightarrow A'$, $B \rightarrow B'$, and $C \rightarrow C'$ saturates the network, an **optimal** solution would be to **block $X \rightarrow X'$** to maximize overall flow.
- However, **this is unfair to X and X'** , who will not have any network access.

Solutions to Resolve This Conflict

1. Load Balancing (Fair Queuing)

- Instead of fully blocking $X \rightarrow X'$ traffic, the network can use **Weighted Fair Queuing (WFQ)**.
- This method **allocates bandwidth fairly** among competing traffic flows while still prioritizing important data.

Example:

- $A \rightarrow A'$, $B \rightarrow B'$, $C \rightarrow C'$ get **70% of the bandwidth**.
- $X \rightarrow X'$ gets **30% of the bandwidth** instead of being shut down.

2. Multipath Routing (Traffic Splitting)

- Instead of forcing all traffic through congested links, traffic can be **split across multiple paths**.

Conclusion

- **Fairness ensures all users get a share of the network**, even if it means reducing efficiency.
- **Optimality aims to maximize throughput**, even if some users get completely blocked.
- **A trade-off is necessary** to ensure practical and balanced network performance.

Would you like an example of how real-world networks handle this balance? 🚀

6. Problem on sink tree

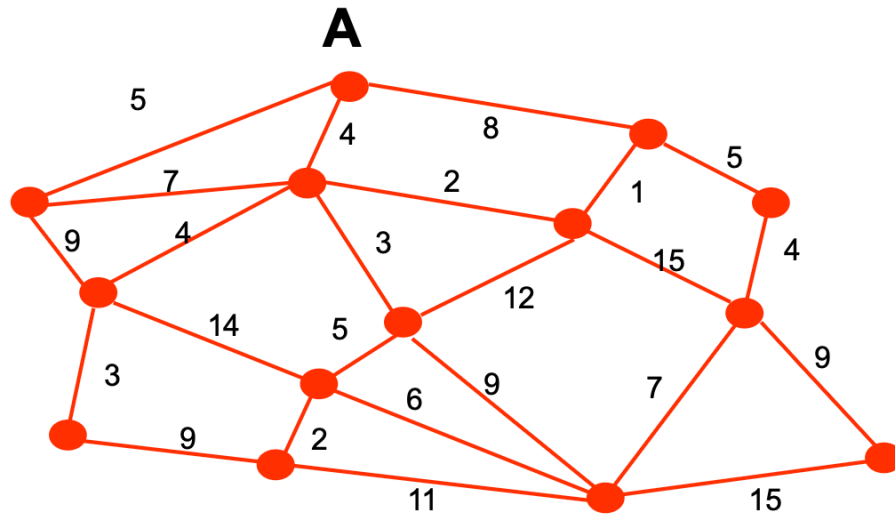
Construct the Sink Tree

- For each node, **connect it to the node from which it received the shortest path**.

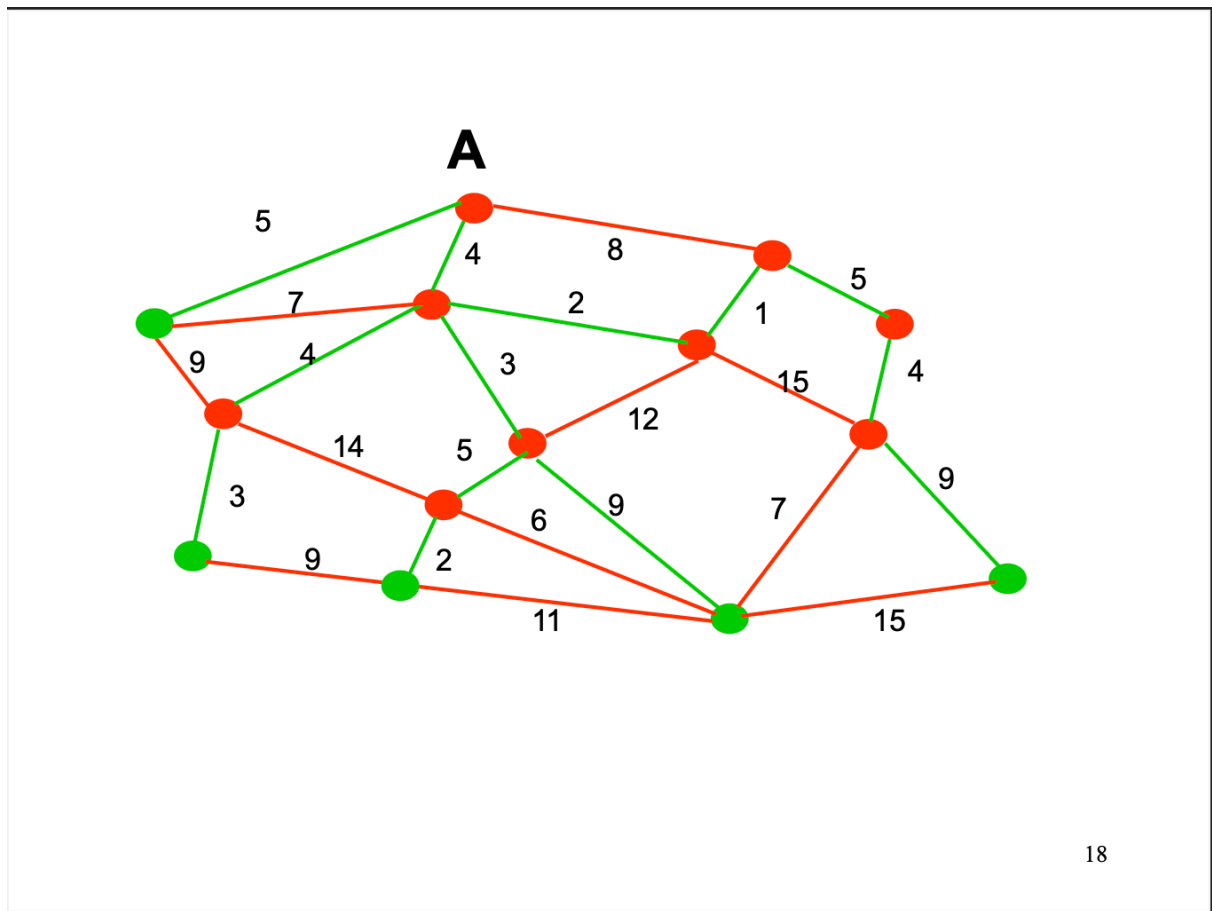
- The resulting structure forms a **directed tree**, where **all edges point toward the sink**.

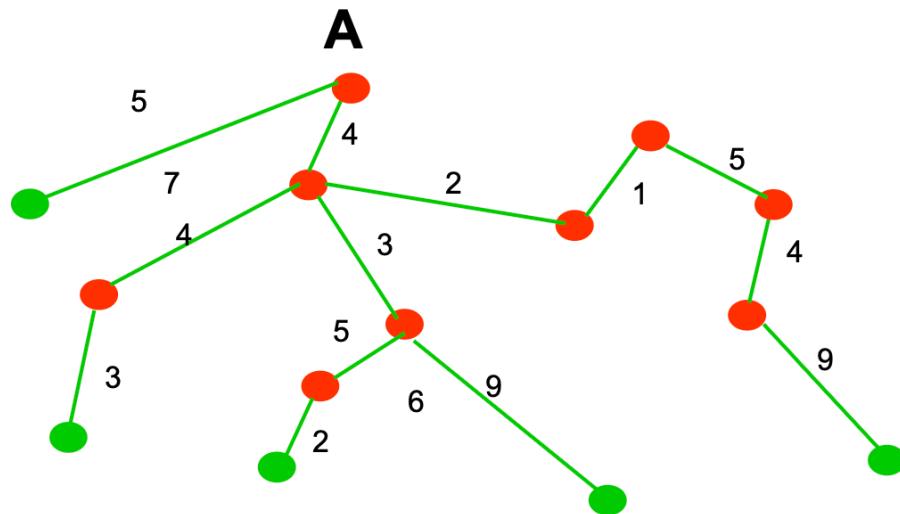
Exercise

Draw the sink tree taking A as the root/destination.



17





19

7. Which parameters are consider in measuring the path distance in routing algorithm

1. Propagation delay
2. BW/Link capacity in bps
3. average traffic load on the link
4. communication cost (in cost of instead of line)
5. mean queue length of the routers along the path

The combination of above criteria the algorithm measure the shortest path

8. How Excess flooding is protected in flooding algorithm

Protection Against Excess Flooding in the Flooding Algorithm

Flooding is a network routing algorithm where each incoming packet is sent to all possible outgoing links except the one it arrived from. However, this can lead to **excessive flooding**, causing congestion and network overload. To prevent this, several mechanisms are used:

1. Hop Count (Time-to-Live - TTL) Limitation

- a. Each packet is assigned a **hop count (TTL)** value.
- b. The value is decremented at every hop.
- c. When the hop count reaches zero, the packet is discarded.
- d. This prevents packets from looping indefinitely.

2. Sequence Numbering

- a. Each packet is assigned a **unique sequence number**.
- b. Nodes maintain a table of recently received sequence numbers.
- c. If a node receives a duplicate packet (same sequence number), it discards it.
- d. Prevents redundant retransmission of the same packet.

3. Selective Flooding

- a. Instead of sending packets to all neighbour's, only those in the **direction of the destination** receive them.
- b. Reduces unnecessary transmissions and saves bandwidth.

4. Hop-Based Threshold

- a. A packet is forwarded only if it comes from a node that is **closer to the destination**.
- b. Ensures controlled packet propagation and avoids unnecessary loops.

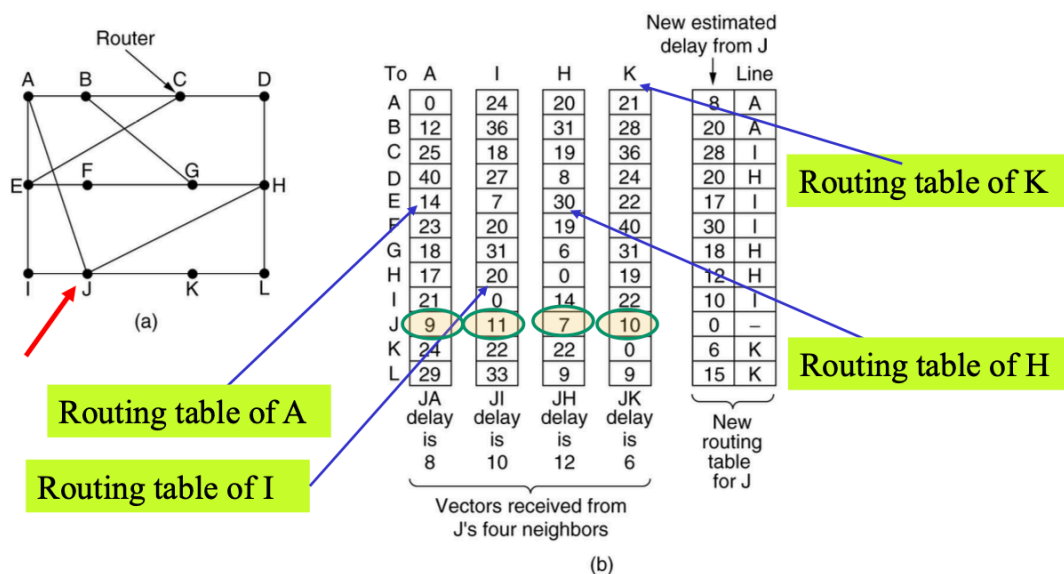
Key Takeaway

Excess flooding can **overload network bandwidth and cause congestion**. By implementing **TTL, sequence numbering, selective flooding, and rate limiting**, networks can **efficiently manage flooding while preventing excessive data transmission**.

Would you like an example of how TTL works in packet forwarding? 🚀

9. Problem on distance vector routing

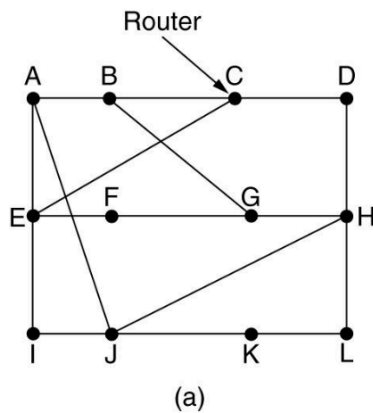
The updating process is illustrated in fig. below considering a subnet. Router **J** has four neighbors **A, I, H, and K**. The first four columns of fig.(b) shows the delay vectors received from the neighbors of router **J**.



(a) A subnet. (b) Input from A, I, H, K, and the new routing table for J.

43

Now assume that some delays are changed shown below the table. Now the new routing table of J will be:



Delay from J to G via A, $JA+AG = 8 + 18 = 26\text{ms}$
 Delay from J to G via I, $JI+IG = 10 + 31 = 41\text{ms}$
 Delay from J to G via H, $JH+HG = 12 + 6 = 18\text{ms}$
 Delay from J to G via K, $JK+KG = 6 + 31 = 37\text{ms}$

Diagram illustrating the process of determining the next hop for a packet from node J to node K.

Left Table: Vectors received from J's four neighbors

To	A	I	H	K
A	0	24	20	21
B	12	36	31	28
C	25	18	19	36
D	40	27	8	24
E	14	7	30	22
F	23	20	19	40
G	18	31	6	31
H	17	20	0	19
I	21	0	14	22
J	9	11	7	10
K	24	22	22	0
L	29	33	9	9

Below the table, the delay for each neighbor is listed:

- JA delay is 8
- JI delay is 10
- JH delay is 12
- JK delay is 6

Right Table: New estimated delay from J

The right table shows the estimated delay from node J to various destinations, calculated as the minimum of the neighbor's delay plus the delay to the neighbor.

Line	Estimated Delay	Next Hop
A	8	A
B	20	A
C	28	I
D	20	H
E	17	I
F	30	I
G	18	H
H	12	H
I	10	I
J	0	-
K	6	K
L	15	K

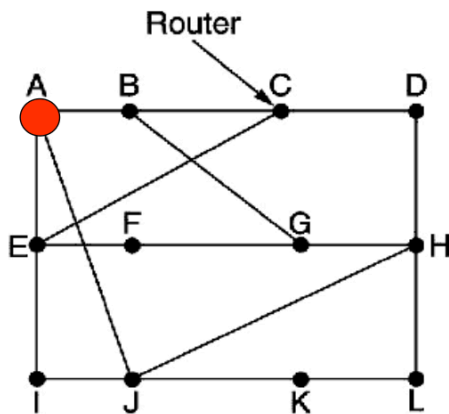
The value 18 for destination G is circled in green, indicating it is the next hop for packets destined to G.

Bottom Label: New routing table for J

10. How distance vector routing response to good and bad news

The count-to-infinity problem

The distance vector routing react rapidly to good news, but leisure to bad news. To see how fast good news propagates, consider the five-node (linear) subnet of fig. below, where the delay metric is the number of hops. Suppose A is down initially and all the other routers know this. In other words, they have all recorded the delay to A as infinity.

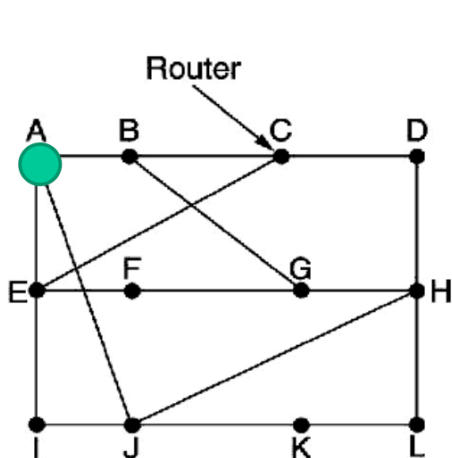


A	B	C	D	H	
•	•	•	•	•	Initially
	1	•	•	•	After 1 exchange
	1	2	•	•	After 2 exchanges
	1	2	3	•	After 3 exchanges
	1	2	3	4	After 4 exchanges

Good news

45

Now let us consider the situation of fig. below in which all the lines and routers are initially up. Routers B, C, D, and E have distances to A of 1, 2, 3, and 4, respectively. Suddenly A goes down, or alternatively, the line between A and B is cut, which is effectively the same thing from B's point of view.



A	B	C	D	H	
•	1	2	3	4	Initially
	3	2	3	4	After 1 exchange
	3	4	3	4	After 2 exchanges
	5	4	5	4	After 3 exchanges
	5	6	5	6	After 4 exchanges
	7	6	7	6	After 5 exchanges
	7	8	7	8	After 6 exchanges
		⋮			
•	•	•	•	•	

Bad News

46

Explanation of the Two Images: Distance Vector Routing and Count-to-Infinity Problem

The two images illustrate how **Distance Vector Routing** responds to **good news (a new shorter route)** and **bad news (a link failure)**. It also highlights the **count-to-infinity problem**, a major issue in Distance Vector Routing.

1. Good News Propagation (First Image)

- **Scenario:** Initially, router A is down, and all other routers have their distance to A as infinity.
 - **Good News:** When A becomes active, the information about its availability spreads quickly.
 - **Reaction Speed:** Distance vector updates are exchanged, and the shortest paths to A are learned in just a few steps.
 - **Conclusion:** The good news spreads rapidly as routers exchange their updated distance vectors efficiently.
-

2. Bad News Propagation & Count-to-Infinity Problem (Second Image)

- **Scenario:** Initially, all routers know the shortest distance to A. Suddenly, A goes down or the link between A and B fails.
- **Bad News Delay:** Unlike good news, bad news takes much longer to propagate due to the **count-to-infinity problem**:
 - When B realizes A is down, it informs its neighbors.
 - However, since C still believes it can reach A via B, C continues to advertise a shorter path.
 - This causes a loop where distances to A gradually increase over time.
 - The process continues until the routers converge to infinity, leading to slow convergence.

Conclusion: Bad news spreads slowly because each router incorrectly assumes its neighbours still have a working route.

Key Takeaways

1. **Distance Vector Routing** reacts quickly to good news but slowly to bad news due to the **count-to-infinity problem**.
 2. **The count-to-infinity problem occurs because routers keep updating their distances based on outdated information from neighbors.**
 3. **Solution:** To fix this issue, modern routing protocols use techniques like:
 - a. **Split Horizon:** Prevents a router from advertising a route back to the router it learned it from.
 - b. **Poison Reverse:** Advertises failed links with infinite cost to speed up convergence.
 - c. **Triggered Updates:** Immediately notify all routers about a broken link.
-

This explanation shows why distance vector routing is **efficient for positive updates** but **problematic for failures**, requiring additional mechanisms for stability. 🚀